

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_11

11. Facade Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Facade pattern.

GoF Definition

It provides a unified interface to a set of interfaces in a subsystem. The Facade pattern defines a higher-level interface that makes the subsystem easier to use.

Concept

This pattern is a great promoter of loose coupling. Using this pattern, you emphasize the abstraction and hide the complex details by exposing a simple interface.

Consider a simple case. Let's say an application has multiple classes and each consists of multiple methods. A client can make a product using a combination of methods from these classes, but they need to remember which classes to pick and which methods to use with the calling sequence of these constructs. It can be ok for a developer, but it makes a client's life difficult. It is also challenging if there are lots of variations among these products.

To overcome this, you can use the Facade pattern. This pattern provides the client a user-friendly, simple interface to use, so that all the inner complexities (like which methods to invoke and what should be the call-

ing sequence or series of steps to follow) are hidden. As a result, a client can simply concentrate on the final outcome(s).

In short, facades provide you with an entry point to access various methods across different classes in a structured manner. If you enforce a rule that does not allow you access to the individual methods directly, and instead you access them through your facade only, then the facade is called an **opaque** facade ; otherwise, it is a **transparent** facade. At the time of this writing, you cannot tag the keyword `static` with a top-level class in Java. The Java Language Specification (Java SE 17 Edition) says that the modifier `static` pertains only to member classes and local classes. But depending on the programming language, you can make a top-level static class. For example, C# allows this, but there are restrictions such as you are allowed to use static data and methods inside the static class. So, depending on the programming language, you can make a top-level static class with static methods to avoid direct instantiation of a facade class. And when you use a similar concept, we say that you are using a **static** facade.

Real-World Example

Suppose you are planning to host a birthday party with 300 guests. Nowadays you can hire a party organizer and let them know the key information such as the party type, date and time of the party, number of attendees, and so on. The organizer will do the rest for you. You do not need to worry about how they will decorate the party room, how they manage the food in buffet style, and so on.

Consider another example. Suppose a customer requests a loan from a bank. In this case, the customer is only interested in knowing whether the loan is approved or not. They do not care about the inner background verification processes conducted at the bank's end. You'll see this as a sample case study in this chapter.

Computer World Example

Think about a case when you use a method from a library (in the context of a programming language). You don't worry about how the method is implemented in the library. You just call the method for easy usage.

You can consider the `java.net.URL` class as an example of a Facade pattern implementation. Here you can see the user-friendly and simplified methods that reduce the inner complexity of interacting with different methods that belong to different classes. Let's examine the concept:

- Investigate the shorthand methods `openStream()` or `getContent()` in this class. The `openStream()` method returns `openConnection().getInputStream()` and the `getContent()` method returns `openConnection().getContent()`. When you see the definition of `openConnection()`, you see that it is another method defined in the `java.net.URL` class and this method returns `handler.openConnection(this)`. What is this handler? It is an `URLConnectionHandler` object. So, you can see that invoking `openStream()` simplifies the invoking of other methods that belong to different classes.
- Similarly, you can consider another method called `setURLConnectionHandlerFactory(...)`. It uses the `URLConnectionHandler` and the `URLConnectionHandlerFactory` instances inside its definition.

The upcoming demonstration will make the overall concept clearer to you.

Implementation

Consider a case when a person applies for a loan from a bank. The bank officials must do some background verification before they grant the loan to the customer. This background verification is a complex process that consists of various subprocesses. The bank officials can visit a customer's property, too, before they consider the loan application. If the applicant satisfies all criteria, he can get a loan. But here is the key: the applicants do not know the details of the background verification. They are interested only in the final outcome: whether they get the loan. The final decision is made by the bank officials only. In the upcoming example, you will see such a process. For simplicity, let's make the following assumptions about approving a loan to a customer:

- The loan applicant or customer must have some assets. If the asset value is less than the loan amount he wants, he cannot get the loan.
- If the customer has an existing loan, he cannot get a new loan.
- If the above points do not apply to a customer, he can get a loan.

Your job is to make an application that is based on these assumptions. Here is sample output for better clarity:

Case 1:

Bob's current asset value: \$5000,
He claims loan amount: \$20,000
He has an existing loan.

Expected Outcome: Bob cannot get the loan for the following reasons:

- Insufficient balance.
- An old loan exists.

Case 2:

Jack's current asset value: \$70,000,
He claims loan amount: \$30,000
He has no existing loan.

Expected Outcome: Jack can get the loan.

Case 3:

Tony's current asset value: \$125000,
He claims loan amount: \$50,000
He has an existing loan.

Expected Outcome: Tony cannot get the loan for the following reason:

- An old loan exists.

Let's build the application now. As per the requirements, in the upcoming example, you see three classes: `Person`, `Asset`, and `LoanStatus`. This class has a constructor that takes three parameters: `name`, `assetValue`, and `previousLoanExist`. Here is the `Person` class:

```
class Person {
```

```

String name;
double assetValue;
boolean previousLoanExist;
public Person(String name, double assetValue, boolean
                previousLoanExist) {

    this.name = name;
    this.assetValue = assetValue;
    this.previousLoanExist = previousLoanExist;
}
}

```

In this client code, you see three loan applicants Bob, Jack, and Tony. They are the instances of `Person` class:

```

// Person-1
Person person = new Person("Bob", 5000, true);
// Person-2
person = new Person("Jack", 70000, false);
// Person-3
person = new Person("Tony", 125000, true);

```

Now see the `Asset` class. This class has the method `hasSufficientAssetValue` to verify whether the current asset value is greater than or equal to the claim amount. Here is the `Asset` class:

```

class Asset {
    public boolean hasSufficientAssetValue(Person person,
                                           double claimAmount) {
        System.out.println("Verifying " + person.name + "'s asset value.");
        return person.assetValue >= claimAmount ? true : false;
    }
}

```

Now see the `LoanStatus` class. This class has the method `hasPreviousLoans` to verify whether a person has an existing loan.

```

class LoanStatus {
    public boolean hasPreviousLoans(Person person) {
        System.out.println("Verifying " + person.name + "'s previous
                           loan(s) status.");
        return person.previousLoanExist ? true : false;
    }
}

```

```
}
```

Notice the conditional operators in the bodies of these methods. You could use an `if-else` chain also. You can use either option to make the methods `hasSufficientAssetValue` and `hasPreviousLoans` work.

Class Diagram

Figure 11-1 shows the class diagram. Notice that the `LoanApprover` class plays the role of the facade. Here you can see the visibility and parameter type(s) of a method too.

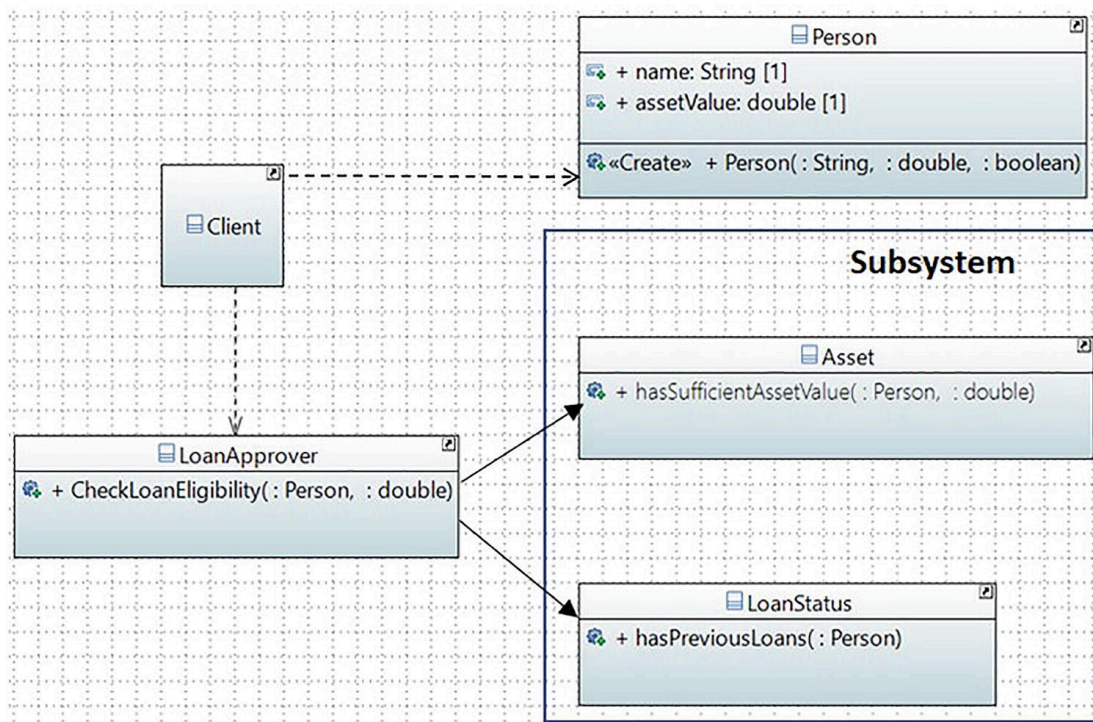


Figure 11-1 Class diagram

Package Explorer View

Figure 11-2 shows the high-level structure of the program. From this view, you can see that at a high level, the subsystem classes (`Asset` and `LoanStatus`) are segregated from the facade class (`LoanApprover`) and the client code.

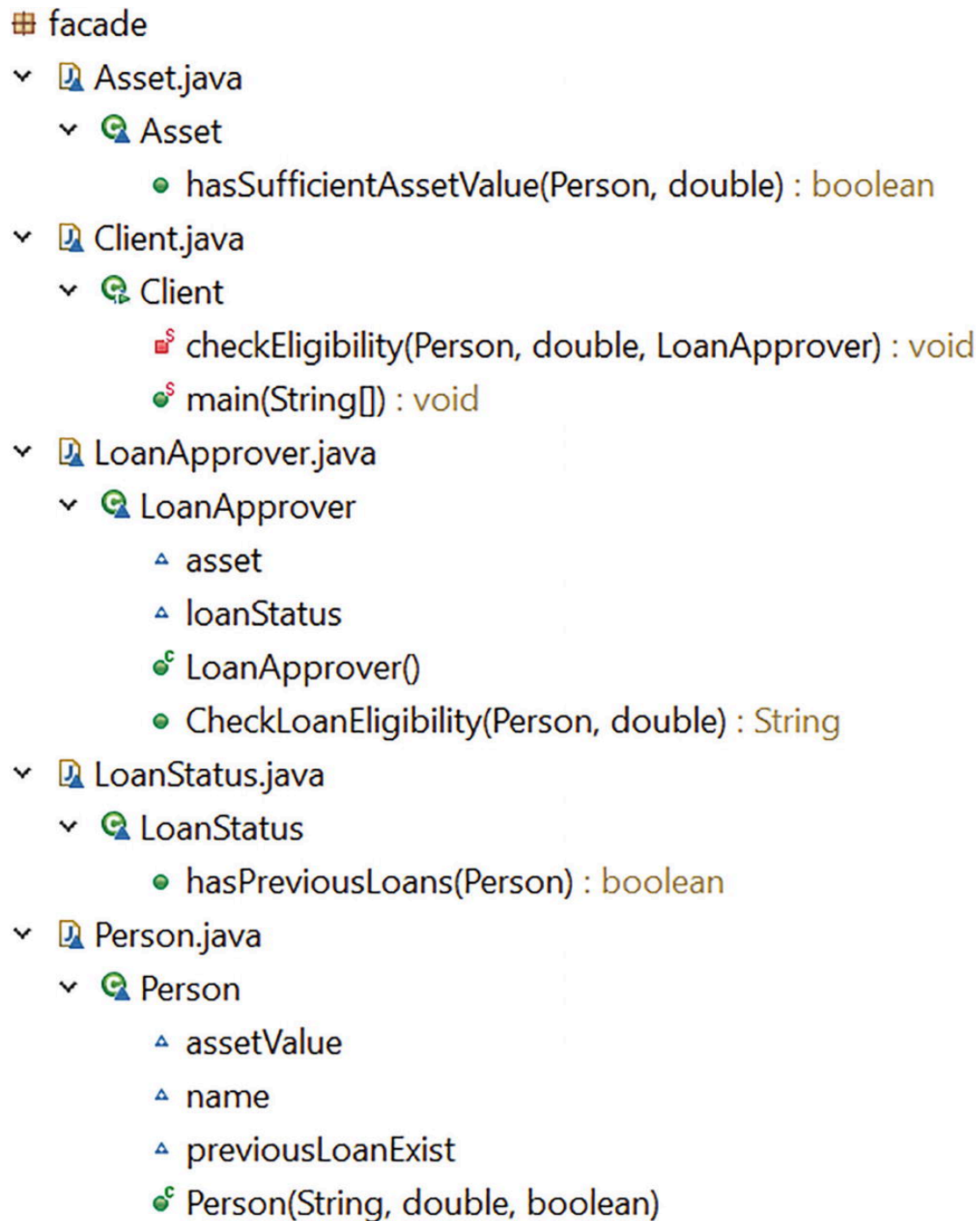


Figure 11-2 Package Explorer view

Demonstration

Here's the full implementation. You can download the code from the `jdp3e.facade` package.

```
// Person.java
class Person {
    String name;
    double assetValue;
    boolean previousLoanExist;
    public Person(String name, double assetValue, boolean
```

```

        previousLoanExist) {

            this.name = name;
            this.assetValue = assetValue;
            this.previousLoanExist = previousLoanExist;
        }
    }

    // Asset.java
    class Asset {
        public boolean hasSufficientAssetValue(Person person,
                                                double claimAmount) {
            System.out.println("Verifying " + person.name + "'s
                                asset value.");
            return person.assetValue >= claimAmount ? true :
                                                                false;
        }
    }

    // LoanStatus.java
    class LoanStatus {
        public boolean hasPreviousLoans(Person person) {
            System.out.println("Verifying " + person.name + "'s
                                previous loan(s) status.");
            return person.previousLoanExist ? true : false;
        }
    }

    // LoanApprover.java
    class LoanApprover {
        Asset asset;
        LoanStatus loanStatus;
        public LoanApprover() {
            asset = new Asset();
            loanStatus = new LoanStatus();
        }
        public String CheckLoanEligibility(Person person,
                                            double claimAmount) {
            String status = " Approved";
            String reason = "";
            System.out.println("\nChecking the loan approval
                                status of " + person.name);
            System.out.println("[The current asset value: " +
                                person.assetValue + ",\n claim

```

```

        amount: " + claimAmount

        + ",\n existing loan?: " +
        person.previousLoanExist + "]\n");
if (!asset.hasSufficientAssetValue(person,
        claimAmount)) {
    status = " Not approved.";
    reason += "\nInsufficient balance.";
}
if (loanStatus.hasPreviousLoans(person)) {
    status = " Not approved.";
    reason += "\nAn old loan exists.";
}
// Collecting the remarks (if any).
String remarks = String.format("%nRemarks if
        any:%s", reason);
// Combining the final result with remarks using
// the format() method.
String result = String.format("%s %s", status, remarks);
// Same as:
// String result=status+"\nRemarks if any:"+reason;
return result;
    }
}

// Client.java
class Client {
    public static void main(String[] args) {

        System.out.println("***Simplifying the usage of a
            complex system using a facade.**");
        // Using a facade
        LoanApprover loanApprover = new LoanApprover();
        // Person-1
        Person person = new Person("Bob", 5000, true);
        checkEligibility(person, 20000, loanApprover);
        System.out.println("-----");
        // Person-2
        person = new Person("Jack", 70000, false);
        checkEligibility(person, 30000, loanApprover);
        System.out.println("-----");
    }
}

```

```

        // Person-3
        person = new Person("Tony", 125000, true);
        checkEligibility(person, 125000, loanApprover);
        System.out.println("-----");
    }
    private static void checkEligibility(
        Person person,
        double claimAmount,
        LoanApprover approver) {
        String approvalStatus =
            approver.CheckLoanEligibility(person, claimAmount);
        System.out.println(person.name + "'s application
                            status:" + approvalStatus);
    }
}

```

Output

Here's the output:

```

***Simplifying the usage of a complex system using a facade.***
Checking the loan approval status of Bob
[The current asset value: 5000.0,
 claim amount: 20000.0,
 existing loan?: true ]
Verifying Bob's asset value.
Verifying Bob's previous loan(s) status.
Bob's application status: Not approved.
Remarks if any:
Insufficient balance.
An old loan exists.
-----
Checking the loan approval status of Jack
[The current asset value: 70000.0,
 claim amount: 30000.0,
 existing loan?: false ]
Verifying Jack's asset value.
Verifying Jack's previous loan(s) status.
Jack's application status: Approved
Remarks if any:
-----
Checking the loan approval status of Tony

```

```
[The current asset value: 125000.0,  
  claim amount: 125000.0,  
  existing loan?: true ]  
Verifying Tony's asset value.  
Verifying Tony's previous loan(s) status.  
Tony's application status: Not approved.  
Remarks if any:  
An old loan exists.  
-----
```

Analysis

This demonstration shows you that a facade provides the following benefits:

- You make a simplified interface for your client, and you reduce the number of objects that a client needs to deal with. Notice that a client does not need to create any subsystem instances (for example, an `Asset` or `LoanStatus` instance) to know the final result. Instead, there is a loan approver instance that is the only point of contact to know the status of an application.
- In the future, if there are any new criteria to get a loan, let the loan approver take the responsibility of handling the situation. I believe that you value this point.
- If there is a big number of subsystems, managing those subsystems with a facade can make communication easier.
- Have you noticed that inside the client code you do not expose your background verification logic? I believe that you value this point too.

Q&A Session

11.1 What are the key advantages of using the Facade pattern?

Here are some advantages:

- If your system consists of many subsystems, managing those subsystems becomes tough, and clients will find it difficult to communicate separately with each of these subsystems. In a case like this, facade patterns are handy. Instead of presenting complex subsystems, you

present one simplified interface to clients. This approach also supports weak coupling by separating the client code from the subsystems.

- It reduces the number of objects that a client needs to access.

11.2 The facade class uses compositions in this example. Is this necessary?

Yes. By using this approach you can access the intended methods in each subsystem.

11.3 Can you access each of the subsystems directly?

Yes, you can. The Facade pattern does not restrict you from doing this. But in this case, the code may look dirty, and you may lose the benefits associated with the Facade pattern. In this context, note that since the client can directly access the subsystem, it is called a **transparent** facade. But when you restrict that usage and force them to use a `LoanApprover` instance only to know whether they can get the loan, it is an **opaque** facade.

11.4 How is the Facade pattern different from the Adapter design pattern?

In the Adapter pattern, you try to alter an interface so that your clients do not see the difference between the interfaces. By contrast, the Facade pattern simplifies the interface. So, a client can interact with a simple interface instead of using a complex subsystem.

11.5 There should be only one facade for a complex subsystem. Is this correct?

Nope. You can create any number of facades for a subsystem.

11.6 Can you add new stuff/additional code with a facade?

Yes, you can. Note the use of simple statements inside the `LoanApprover` class to enable you to see messages like the following:

```
Checking the loan approval status of Jack
[The current asset value: 70000.0,
 claim amount: 30000.0,
```

```
existing loan?: false ]
```

11.7 What are the challenges associated with this pattern?

Here are some challenges:

- Subsystems are connected to the facade layer. So, you need to take care of an additional layer of coding (thus increasing your codebase).
- When the internal structure of a subsystem changes, you need to incorporate the changes in the facade layer also.
- Developers need to learn about this new layer, whereas some of them are aware of how to use the subsystems/APIs efficiently.

11.8 In a different programming language such as C#, I have seen static facade classes. Can I make the same in Java?

In many examples, there is only one facade and you may not need to initialize the facade class. In those cases, it makes sense if you make the facade class static. But remember that Java does not allow you to create a top-level static class, whereas in C#, it is allowed but there are restrictions.

You can refer to the Java 16 language specification in this context at <https://docs.oracle.com/javase/specs/jls/se16/jls16.pdf> which says the following:

The modifier static pertains only to member classes and local classes. It further repeats the same: **It is a compile-time error if a local class or interface declaration has the modifier static (§8.1.1)**

Note Java 17 is released now. This language specification tells the same. This specification is available at the following link: <https://docs.oracle.com/javase/specs/jvms/se17/jvms17.pdf>.

Previous chapter

< [10. Adapter Pattern](#)

Next chapter

[12. Flyweight Pattern](#) >